Markov chains and n-grams

Computational modeling of language
Spring 2026

Possible relations of structure and numbers

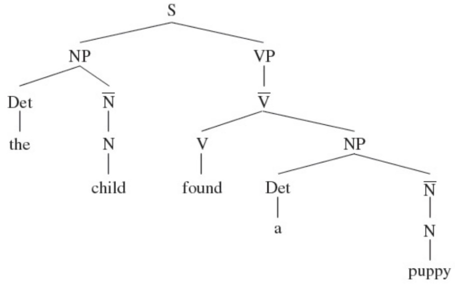
Arranged in a cline of increasing importance for numbers

- All symbols, **no** numbers
- Symbols **augmented** by numbers
- Numbers as **content providers** for symbols
- **All** numbers, no symbols

Possible relations of structure and numbers

Arranged in a cline of increasing importance for numbers

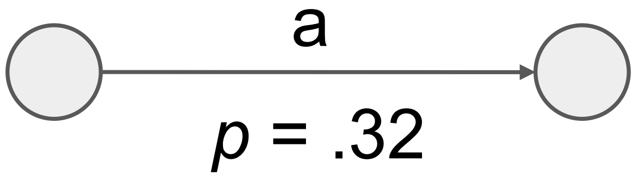
- All symbols, **no** numbers
- Symbols **augmented** by numbers
- Numbers as **content providers** for symbols
- **All** numbers, no symbols



Possible relations of structure and numbers

Arranged in a cline of increasing importance for numbers

- All symbols, **no** numbers
- Symbols **augmented** by numbers
- Numbers as **content providers** for symbols
- **All** numbers, no symbols



Markov chains

An extension of probability theory in what was then (1913) a new direction: from **independent** coin flips to **dependent** events in a series.

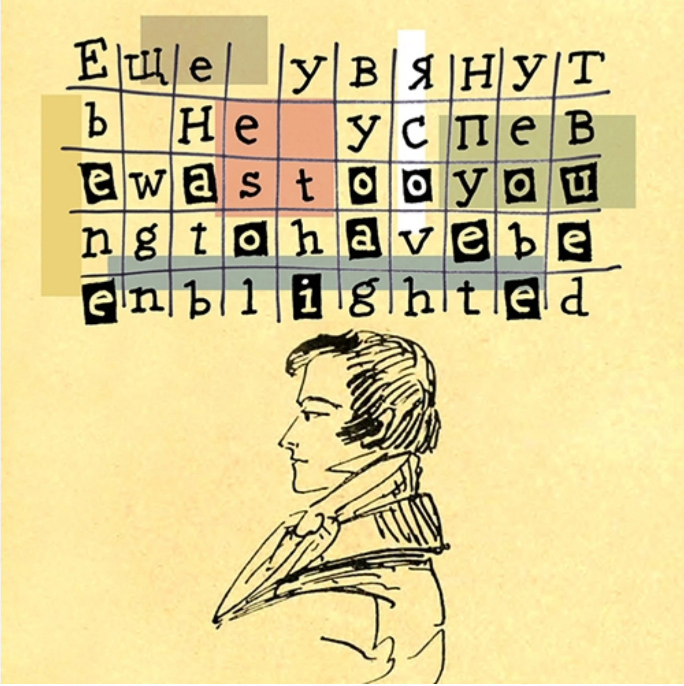
Hayes 2013

Markov chains

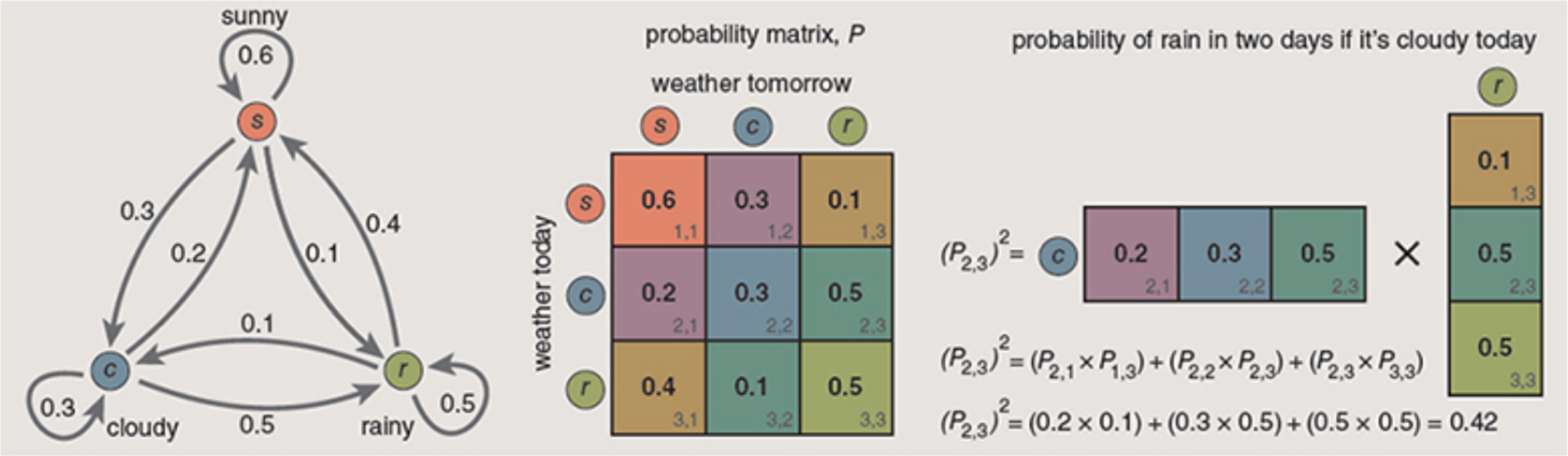
An extension of probability theory in what was then (1913) a new direction: from **independent** coin flips to **dependent** events in a series.

First illustrated in application to **language**!

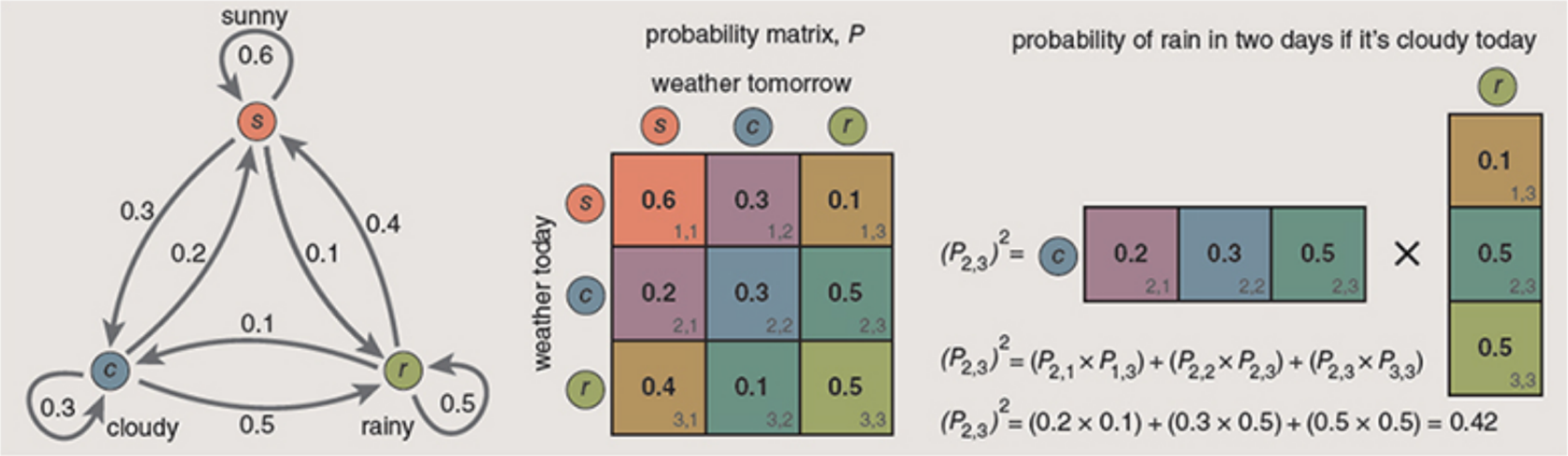
Hayes 2013



Hayes 2013



A simple non-linguistic example:
Predicting tomorrow's weather from today's.



Markov assumption

The probabilities must depend only on the **present state** of the system, **not on its earlier history**.

Andrei Andreevich Markov
(1856 - 1922)

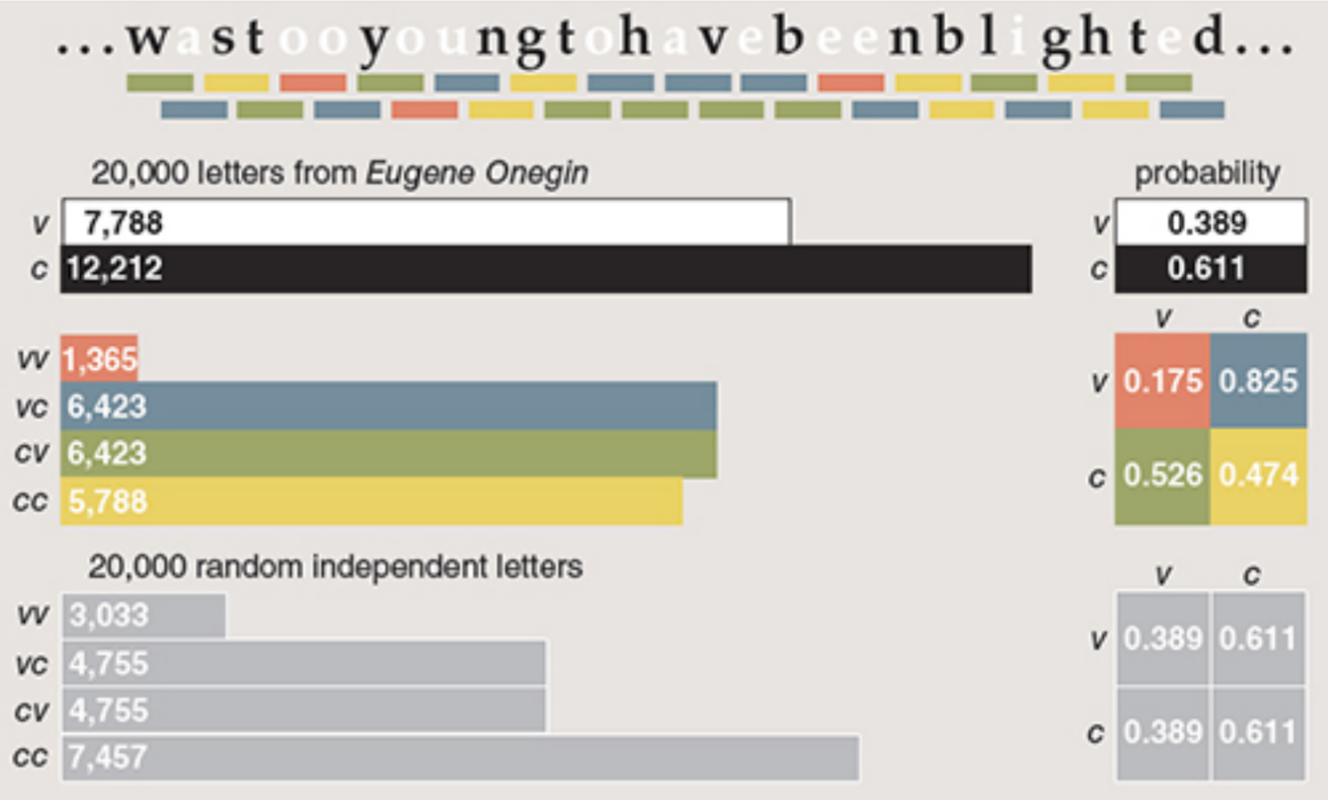
Andrei Andreevich Markov
(1856 - 1922)

His son, also named Andrei Andreevich Markov, became a distinguished mathematician of the Soviet era ...

Andrei Andreevich Markov
(1856 - 1922)

His son, also named Andrei Andreevich Markov, became a distinguished mathematician of the Soviet era ...

To the consternation of librarians, both father and son signed their works “A. A. Markov.”



Markov's original example:
Vowels and consonants in Pushkin's *Eugene Onegin*.

Hayes estimates it must have taken Markov **several days** to count up the numbers by hand.

Hayes estimates it must have taken Markov **several days** to count up the numbers by hand.

Today it can be done in **milliseconds** - as you will find out in your homework.

Hayes estimates it must have taken Markov **several days** to count up the numbers by hand.

Today it can be done in **milliseconds** - as you will find out in your homework.

Compare **Greenberg** - a corpus linguist before his time - who also counted by hand, or relied on others doing so, things that we would nowadays use Pandas for and get results more or less instantaneously.

TABLE XXXIII

G^{+1}	father	3235	padre	5631
	mother	3993	madre	5598
G^{-1}	son	993	hijo	3765
	daughter	865	hija	1749
	child	1574		
G^{0}	brother	659	hermano	3120
	sister	590	hermana	1811
G^{+2}	grandfather	173	abuelo	1234
	grandmother	346	abuela	1540
G^{-2}	grandson	32	nieto	94
	granddaughter	33	nieta	58
G^{+3}	great-grandfather	8	bisabuelo	83
	great-grandmother	19	bisabuela	10
G^{-3}	great-grandson	0	bisnieto	4
	great-granddaughter	0	bisnieta	4

Greenberg, J.H. (1980). *Language universals*, p. 80.

N-gram language models

"You are uniformly charming!" cried he, with a smile of associating and now and then I bowed and they perceived a chaise and four to wish for.

Random sentence generated from a Jane Austen trigram model.

Jurafsky & Martin

Language model

A model that assigns a **probability** to a sequence of words.

For example, intuitively:

p("*he briefed reporters on the main contents of the statement*")

>

p("*he introduced reporters to the main contents of the statement*")

Jurafsky & Martin

N-gram model

The **simplest** language model - based on a fixed-length **strip of words**, or an **n-gram**. For example, a trigram contains 3 words:

For this reason, we'll need to introduce cleverer ways of estimating

N-gram model

The **simplest** language model - based on a fixed-length **strip of words**, or an **n-gram**. For example, a trigram contains 3 words:

For $\boxed{\text{this reason, we'll}}$ need to introduce cleverer ways of estimating

N-gram model

The **simplest** language model - based on a fixed-length **strip of words**, or an **n-gram**. For example, a trigram contains 3 words:

For this reason, we'll need to introduce cleverer ways of estimating

N-gram model

The **simplest** language model - based on a fixed-length **strip of words**, or an **n-gram**. For example, a trigram contains 3 words:

For this reason, $\boxed{\text{we'll need to}}$ introduce cleverer ways of estimating

N-gram model

The **simplest** language model - based on a fixed-length **strip of words**, or an **n-gram**. For example, a trigram contains 3 words:

For this reason, we’ll need to introduce cleverer ways of estimating

Use counts to determine *p*(“*to*” | “*we’ll need*”)

N-gram model

The **simplest** language model - based on a fixed-length **strip of words**, or an **n-gram**. For example, a trigram contains 3 words:

For this reason, $\boxed{\text{we'll need to}}$ introduce cleverer ways of estimating

Use counts to determine *p*("*to*" | "*we'll need*")
More generally, determine *p*(word | its fixed-length history)

Wait!

This is just wrong! The probability of a word appearing could be affected by a long-distance dependency!

Wait!

This is just wrong! The probability of a word appearing could be affected by a long-distance dependency!

True: our attending only to the recent past is a matter of **convenience** or tractability, not principle.

Wait!

This is just wrong! The probability of a word appearing could be affected by a long-distance dependency!

True: our attending only to the recent past is a matter of **convenience** or tractability, not principle.

In fact in our homework we will be examining **bigrams**, i.e. we will compute $p(w_n \mid w_{n-1})$.

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)}$$

Or equivalently:

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

Jurafsky & Martin

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Figure 3.1 Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Zero counts are in gray.

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

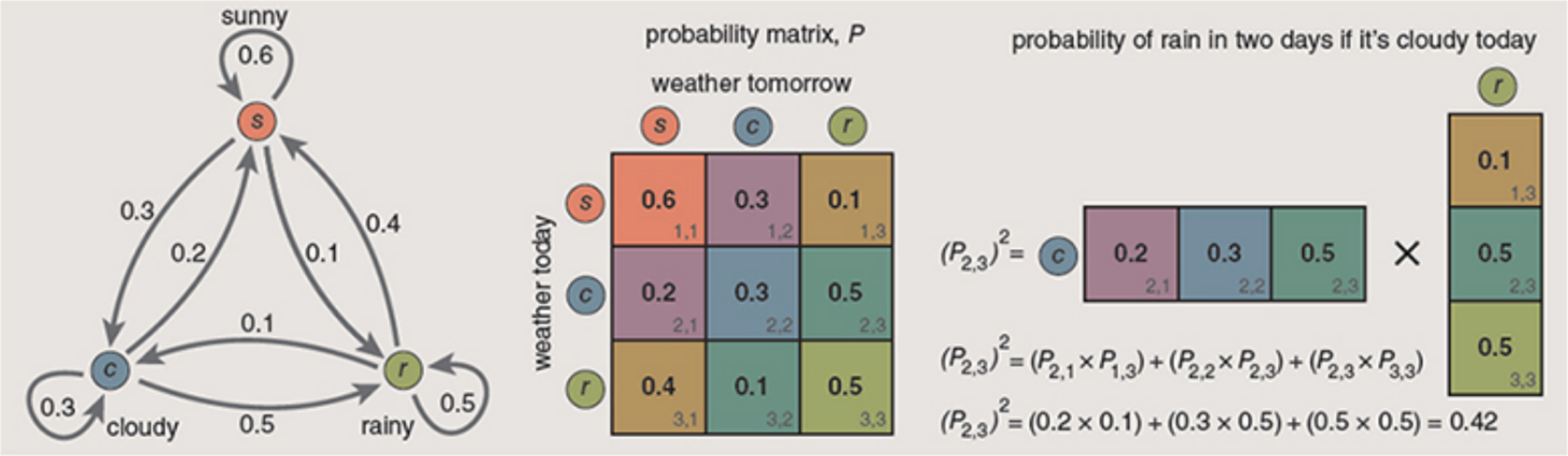
Figure 3.2 Bigram probabilities for eight words in the Berkeley Restaurant Project corpus of 9332 sentences. Zero probabilities are in gray.

$p(want \mid i) =$

$= c(i\ want) / c(i) =$

$= 827 / (5+827+9+2+\ldots)$

$= 0.33$



sunny
0.6
0.3
0.2
0.1
0.4
0.1
0.3
cloudy
0.5
rainy
0.5
probability matrix, P
weather tomorrow
weather today
probability of rain in two days if it's cloudy today
$(P_{2,3})^2 = (P_{2,1} \times P_{1,3}) + (P_{2,2} \times P_{2,3}) + (P_{2,3} \times P_{3,3})$
$(P_{2,3})^2 = (0.2 \times 0.1) + (0.3 \times 0.5) + (0.5 \times 0.5) = 0.42$

$$p(w_n \mid w_{n-1})$$